

RhirePro — Developer Task Plan & Test Guide

Audience: the developer verifying and completing the work. Goal: make RhirePro ready to apply to Google for Startups Cloud (Start tier) and NVIDIA Inception, with every AI claim true.

Repo state: branch main (and `claude/dreamy-clarke-511efn`). Stack: React/Vite frontend, Supabase (Postgres + RLS), Netlify Functions (`/netlify/functions`, helpers in `/netlify/shared`), OpenAI for AI, Razorpay payments.

Rules for every task: audit before building (older notes saying "no jobs/applications/billing backend" are wrong); keep model calls under Netlify's 10 s limit; authenticate + authorise + rate-limit every AI endpoint; never expose `OPENAI_API_KEY` to the browser; run `npm run vitest` and `npm run vite build` before each commit; do not break payment arithmetic (pinned by tests); only claim features that are live.

PART 1 — DAY 1 TEST GUIDE (what is already built)

Placeholders: SITE = your live Netlify URL (e.g. <https://yoursite.netlify.app>). TOKEN = a logged-in user's access token (see 1.0). Replace USER_ID, JOB_ID.

1.0 Get a test token (browser)

Log in as a **job seeker** on the live site, open DevTools → Console, run:

```
JSON.parse(localStorage.getItem(Object.keys(localStorage).find(k => k.includes('auth-token')))).access_token
```

Copy the string. Do the same later as a **recruiter** for recruiter-only tests. The same console snippet with `.user.id` instead of `.access_token` gives USER_ID.

1.1 Automated checks (local)

```
npm ci
npx vitest run      # expect: 19 files, 345 tests passed
npx vite build      # expect: "built in ..." with no errors
```

PASS = both succeed.

1.2 Database checks (Supabase → SQL Editor)

#	Run	Expected
D1	<code>select extname from pg_extension where extname='vector';</code>	1 row
D2	<code>select table_name, column_name from information_schema.columns where column_name in ('embedding', 'embedding_hash', 'embedding_updated_at') and table_name in ('profiles', 'jobs');</code>	6 rows
D3	<code>select proname from pg_proc where proname in ('check_rate_limit', ' purge_rate_limits', 'match_jobs', 'match_applicants');</code>	4 rows
D4	<code>select * from check_rate_limit('test:1', 3, 60);</code> run it 4 times	calls 1-3: allowed = true; call 4: allowed = false, retry_after > 0
D5	<code>delete from rate_limits where key='test:1';</code>	cleanup

1.3 Rate limiting (live endpoints)

Use a made-up email so no real user is affected. `verify-otp` allows 10 attempts per 15 min per email.

```
for i in $(seq 1 12); do
  curl -s -o /dev/null -w "$i -> %{http_code}\n" -X POST "$SITE/api/verify-otp" \
    -H "Content-Type: application/json" \
    -d '{"email":"ratelimit-test-123@example.com","otp":"000000","user_type":"jobseeker"}'
done
```

PASS = attempts 1-10 return a normal 4xx (400/401/404), attempts 11-12 return **429** with a `Retry-After` header. (Wait 15 min or change the email to rerun.)

Also confirm the security headers:

```
curl -sI "$SITE" | grep -i -E "strict-transport|x-content-type|x-frame|referrer-policy"
```

PASS = all four headers present.

1.4 AI endpoints (need OPENAI_API_KEY)

T1 — ai-insights (public)

```
curl -s -X POST "$SITE/api/ai-insights" -H "Content-Type: application/json" -d '{"skills":["React","TypeScript"]}'
```

PASS = HTTP 200, JSON `{"text":{"...trendingSkills...certifications..."}}`.

FAIL "not configured" = key name/scope/redeploy problem. FAIL "temporarily unavailable" = check Netlify function logs (401 = bad key, 429 = no OpenAI credit).

T2 — match-score (seeker applies to a real job id)

```
curl -s -X POST "$SITE/api/match-score" -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
  -d '{"profile_id":"USER_ID","job_id":"JOB_ID"}'
```

PASS = 200 with score (0-100), summary, strengths, gaps, cached:false. Run again → cached:true.

T3 — embed-sync (profile)

```
curl -s -X POST "$SITE/api/embed-sync" -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"kind":"profile","id":"USER_ID"}'
```

PASS = {"status":"updated"}; a second call → {"status":"unchanged"}. A nearly empty profile → {"status":"skipped"} (fill headline/skills/about and retry). Verify in SQL:

```
select id, embedding_updated_at, vector_dims(embedding) from profiles where id='USER_ID'; → vector_dims = 768.
```

T4 — embed-sync (job) — use the **recruiter** token and one of their job ids:

```
-d '{"kind":"job","id":"JOB_ID"}'
```

PASS = updated. Embed at least 3-5 active jobs so search has data. Verify: select count(*) from jobs where embedding is not null;

T5 — semantic-search

```
curl -s -X POST "$SITE/api/semantic-search" -H "Authorization: Bearer $TOKEN" -H "Content-Type: application/json" \
-d '{"mode":"jobs","query":"remote react developer"}'
```

PASS = {"results":[{"job_id":"...", "similarity":0.xx}, ...]} sorted high to low, only Active jobs. Then {"mode":"jobs"} with no query (ranks by the seeker's own profile; needs T3 done) → results, or 409 no_profile_embedding if profile not embedded.

Recruiter ranking: recruiter token, {"mode":"applicants", "job_id":"JOB_ID"} → applicants of that job ranked (needs job + applicant profiles embedded).

T6 — resume-parse (PDF only). Find a real path: select name from storage.objects where bucket_id='resumes' limit 5; (format USER_ID/resume.pdf), then call with that user's token:

```
-d '{"path":"USER_ID/resume.pdf"}' # to $SITE/api/resume-parse
```

PASS = 200 with parsed containing name, skills, experience, education. Takes ~3-8 s.

1.5 Security / negative tests (all must be rejected)

Test	Expected
Any AI endpoint with no Authorization header (except ai-insights)	401
embed-sync {"kind":"profile","id":"<someone else's id>"}	403
embed-sync {"kind":"banana","id":"x"}	400
resume-parse path of another user (OTHER_ID/resume.pdf)	403
resume-parse path USER_ID/resume.docx	415 unsupported_type
resume-parse path containing . .	403
semantic-search {"mode":"applicants", "job_id":"<not your job>"} as a seeker	403
61st embed-sync call within an hour by one user	429

1.6 Observability

- Netlify → Logs → Functions: open each function; no unhandled errors.
- Supabase: select function_name, status_code, count(*) from api_request_logs where created_at > now() - interval '1 hour' group by 1,2; (auth functions log here).
- OpenAI dashboard: usage shows embedding + chat requests.

1.7 Day 1 sign-off

All of 1.1-1.5 pass → Day 1 complete. Record results as a table (test id, PASS/FAIL, notes) and attach to the PR.

PART 2 — DAY-BY-DAY TASK PLAN

Each task: **what** / **where** / **done when**. Update the status table in docs/GRANT_READINESS_PLAN.md after each.

DAY 1 (Monday) — Foundation + audit

- **D1-1 Run and verify the 2 SQL files and env vars.** Done when Part 1 passes.
- **D1-2 Audit Track A (verdict DONE / PARTIAL / MISSING with file evidence)** for: job create/edit/pause/close (RecruiterDashboard, jobService.ts); apply/withdraw/status (JobSeekerDashboard, applications); recruiter candidate search (Candidates tab — real data or mock?); notifications (in-app + email triggers); Razorpay order → verify → plan activation → usage limits → cancel; super-admin moderation. Done when a table of verdicts exists in docs/AUDIT.md.
- **D1-3 Fix every mock-data screen found.** Done when no dashboard renders hard-coded sample arrays.

DAY 2 (Tuesday) — Make the AI visible

- **D2-1 Resume parsing UI.** Profile screen: "Parse my resume" → calls /api/resume-parse with <profile.id>/resume.pdf → review screen with a checkbox per section (headline, skills, experience, education, certifications, about) → saves only what the user confirms. Handle errors: unsupported type, too large, rate-limited, AI down. Done when a real PDF populates the profile after confirmation and nothing is saved without it.
- **D2-2 Embedding sync.** After profile save and after job create/edit, call /api/embed-sync fire-and-forget (never block or fail the save). Done when embedding_updated_at changes after an edit.
- **D2-3 Backfill script** scripts/backfill-embeddings.mjs: embeds all existing profiles/jobs in batches with a delay, resumable, logs progress. Done when count(*) where embedding is null (for profiles with enough content and active jobs) is 0.
- **D2-4 Smart search.** Search bar gets a "Smart search" mode → /api/semantic-search {mode:"jobs", query} → hydrate via existing RLS queries → show a "Best match" label; fall back to keyword search on any error. Done when "remote react jobs with good work-life balance" returns sensible jobs.
- **D2-5 Recommendations + applicant ranking.** Seeker "Recommended for you" via semantic-search without a query; recruiter applicant list gets "Best match" sort via {mode:"applicants"} with on-demand /api/match-score explanation. Done when both show ranked, explained results.

DAY 3 (Wednesday) — Coverage + hardening + compliance

- **D3-1 DOCX resume support** in /api/resume-parse (server-side docx→text, e.g. mammoth; 4 MB limit). Done when a .docx parses like a PDF.
- **D3-2 AI job-description generator.** New /api/generate-job-description (title + 3-6 bullets → structured draft the recruiter edits; never auto-publishes). Auth + rate limit + server-built prompt.
- **D3-3 Content-Security-Policy** as Content-Security-Policy-Report-Only in netlify.toml (allow Razorpay, Supabase, Google sign-in); fix violations seen in console; then switch to enforcing only if clean.
- **D3-4 Input validation audit** of every Netlify function (type/length checks, no over-broad service-role access) and HTML-rendered user text sanitisation. Done when findings are fixed and listed in docs/AUDIT.md.
- **D3-5 Privacy Policy + Terms** updated: resumes/profiles are processed by a third-party AI provider (OpenAI) for parsing, matching, search; retention, deletion and user rights. Flag for legal review.
- **D3-6 Verify Supabase refresh-token rotation / reuse detection** is enabled (Auth settings); document it.
- **D3-7 Deploy to production and re-run Part 1.**

DAY 4 (Thursday) — Trust & story

- **D4-1 Fraud / fake-job detection** on job create: rules (payment requests, off-platform contact, suspicious domains, near-duplicate via embedding similarity) + LLM judgement → risk score and reasons in a new table; high risk goes to a super-admin moderation queue instead of auto-publishing.
- **D4-2 AI career assistant** (only if D4-1 is done): chat for seekers grounded in their profile + matched jobs (match_jobs), history capped, rate-limited, refuses off-topic, never reveals other users' data.

- **D4-3 Landing page copy** describes only live AI features + a short "How our AI works" section.
- **D4-4** docs/NVIDIA_ROADMAP.md: planned video-interview analysis with NVIDIA Riva/NIM, GPU needs, cost estimate — clearly labelled planned, **not** built.
- **D4-5** docs/PITCH_DECK.md + docs/APPLICATION_ANSWERS.md (Google for Startups and NVIDIA Inception answers) from verified repo facts only; unknown numbers marked [FOUNDER TO FILL].

DAY 5 (Friday) — QA and apply

- **D5-1 End-to-end demo path** on production: sign up → upload resume → parse → confirm → smart search → apply → recruiter ranks applicants → match explanation → premium purchase with a test coupon. Write docs/QA_CHECKLIST.md with results; fix any break.
- **D5-2 Regression:** npx vitest run, npx vite build, RLS check (scripts/verify_ribs_security.mjs), Part 1 tests.
- **D5-3 Founders submit** Google for Startups (Start tier) first, then NVIDIA Inception.

Founder-only items (developer cannot do these)

Incorporation date/entity/registration number; live site + email on the company domain; Google Cloud billing account ID; traction numbers; founder bios; verify current program terms on the official NVIDIA and Google pages.

Cut order if time runs short

First cut: D4-2 (assistant), D4-1 (fraud), D3-2 (JD generator). Never cut D2-1 through D2-5; they are what make "AI-matched" true.